

Ori

Livro-laboratório — ideia, processo, prática e consulta

Versão do livro: **0.3.x-book.2**

Âncora da linguagem: Ori 0.3.x (superfície S3)

Rascunho vivo · não substitui a spec normativa

Índice (ordem do PDF)

01-problema-leitura
02-o-que-e-ori
03-para-que-existe
04-mapa-ecossistema
05-da-ideia-ao-s3
06-pipeline-compilador
07-decisoes-de-design
08-ferramentas
09-ia-no-desenvolvimento
10-instalar
11-anatomia-programa
12-tipos-ausencia-falha
13-expressoes-fluxo-pipe
14-structs-enums-funcoes
15-modulos-projetos
16-traits
17-stdlib-io-async-testes
18-cheat-sheet-sintaxe
19-cli-e-env
20-stdlib-indice
21-diagnostics
22-estabilidade-abi-limites
A-cheat-sheet
B-mapa-repositorio
C-exercicios
D-programa-30-minutos
E-bibliografia
glossario
exemplos-canonicos
README

Cap. 1 — O problema da leitura de código

Versão âncora: Ori 0.3.x (S3)

Parte: I

TL;DR

Código é lido muitas vezes mais do que é escrito. Linguagens que escondem contratos (tipos, falhas, donos de recurso) cobram pedágio cognitivo a cada leitura — especialmente para quem já opera com memória de trabalho limitada.

Exemplo

Dois contratos mentais. Só o da direita deixa a falha **visível na assinatura**:

Opaco para o leitor	Legível na fonte
<code>load(path) -> string</code> — falha?	<code>load(path: string) -> result[string, string]</code>

```
module app.demo

import ori.fs = fs

load(path: string) -> result[string, string]
  return fs.read_text(path)
end
```

Sem `result`, o leitor precisa lembrar (ou adivinhar) se a função pode falhar.

Como funciona

- **Leitura > digitação.** A Ori otimiza o custo de *entender*, não o de digitar menos caracteres.
- **Perguntas do leitor** devem ter resposta na fonte:
 - Onde este arquivo mora? → `module`
 - O valor pode faltar? → `optional[T]`
 - A operação pode falhar? → `result[T, E]`
 - Quando o recurso some? → `using`
- **Neurodivergência** (TDAH, dislexia, autismo, etc.) não é um “extra”: menos regras ocultas e cadeias de inferência longas reduz carga cognitiva para todos.
- Design de linguagem é também design de **acessibilidade cognitiva**.

O que memorizar

- Um programa bom é um programa **fácil de reler**.
- Contrato visível no ponto de uso > magia conveniente.
- A Ori nasceu deste problema — não de uma disputa de market share.

Ir mais fundo

- Manifesto: `../../../../spec/00-manifesto.md` (`../../../../spec/00-manifesto.md`)
- Cap. 3 — finalidade e anti-objetivos

Cap. 2 — O que é Ori

Versão âncora: Ori 0.3.x (S3)

Parte: I

TL;DR

Ori é uma linguagem **compilada AOT**, tipada, com superfície de leitura **S3**, runtime com **ARC**, codegen nativo (Cranelift) e **JIT** opcional em `ori run`. Arquivos `.orl`, CLI `ori`, stdlib `ori.*`.

Exemplo

```
module app.hello

import ori.io = io

main()
  io.println("Hello, Ori!")
  const answer: int = 21 * 2
  io.println(f"The answer is {answer}")
end
```

```
ori run main.orl
```

Como funciona

Camada	O que é
Pele (S3)	Ritmo de leitura: <code>module</code> , blocos <code>end</code> , tipos com <code>[]</code> , traits <code>apply / use</code>
Motor	Async, traits semânticos, ARC, backends nativo (+ C debug), stdlib
Ferramenta	Compilador em Rust; package de release sem exigir Rust no PATH do usuário

Pipeline resumido:

```
.orl → lexer → parser → resolver → types → HIR → Cranelift → binário ou JIT
```

Pré-1.0: a superfície **pode** mudar com disciplina; sintaxe pré-S3 é **erro duro**.

O que memorizar

- Extensão `.orl`, comando `ori`, módulos stdlib `ori.X`.

- AOT para `compile / test`; JIT por padrão em `run` quando há `cdylib`.
- S3 desde `0.3.0`; inferência local opção B desde `0.3.1`.

Ir mais fundo

- Overview normativo: `../../spec/01-overview.md` (`../../spec/01-overview.md`)
- README do repo: `../../..//README.md` (`../../..//README.md`)
- Cap. 6 — pipeline completo

Cap. 3 — Para que existe (e para que não)

Versão âncora: Ori 0.3.x (S3)

Parte: I

TL;DR

Ori existe para **estudar compiladores, explorar programação com IA** no mesmo código e **reduzir carga cognitiva na leitura**. Não existe para substituir Rust, Go ou TypeScript no mercado.

Exemplo

Três perguntas. Se a resposta for “sim” nas três, você está no público certo:

1. Dá para ensinar um pipeline real de compilador com este repo?
2. Dá para um humano e um agente trabalharem no mesmo codebase com regras claras?
3. O código deixa contratos (falha, ausência, módulo) visíveis ao leitor?

Como funciona

Os três pilares

Pilar	O que significa na prática
Estudo	Lexer, tipos, codegen, runtime e stdlib no mesmo monólito — lab sério, não toy
Humano + IA	Skills, AGENTS.md, testes e CHANGELOG como contrato de sessão
Legibilidade ND	Menos regras ocultas; diagnostics com código; contratos na fonte

ori (אורי) — hebraico para “minha luz.” O nome aponta para clareza, não para marketing.

Três leitores típicos

1. **Estudante de compiladores** — quer ver AOT de verdade (Cranelfit, link, ARC), não só AST de brinquedo.
2. **Quem programa com agentes** — quer limites explícitos: o que a IA pode mudar e o que é norma.
3. **Quem se cansa de código opaco** — quer optional / result / module visíveis sem caçar convenções.

Para que *não* serve

Não prometa	Por quê
“Substitui Rust/Go/TS”	Fora do manifesto
Estabilidade 1.0 hoje	Ainda é 0.3.x (FREEZE-1 com disciplina)
Foco em loja de extensão / marketing multi-OS	Produto agora: linguagem, stdlib, docs, performance, DX local
Self-hosting como próximo passo	M4 é a última discussão tática

Uso em projetos **pequenos e médios** é bem-vindo como laboratório — não como campanha comercial.

Como o livro usa esses pilares

- Parte I = por quê
- Parte II = como o lab é construído (inclui Cap. 9 sobre IA)
- Parte III = escrever Ori legível
- Parte IV = consultar sem reabrir a spec inteira

O que memorizar

- Três pilares: estudo · IA · legibilidade ND.
- Anti-pitch: sem guerra de market share.
- Nome = luz = clareza na leitura.

Ir mais fundo

- Manifesto: `../../spec/00-manifesto.md` (`../../spec/00-manifesto.md`)
- Cap. 1 — problema da leitura
- Cap. 9 — IA no desenvolvimento
- Cap. 22 — estabilidade e limites

Cap. 4 — Mapa mental do ecossistema

Versão âncora: Ori 0.3.x (S3)

Parte: I

TL;DR

O repositório é um monólito organizado: compilador Rust, runtime, stdlib `.orl`, spec, exemplos, extensões de editor e ferramentas de release/QA. Cada pasta tem um dono claro.

Exemplo

Árvore mínima para se orientar:

```
ori-lang/  
├─ compiler/crates/    # ori-lexer ... ori-driver, ori-lsp, ori-runtime  
├─ stdlib/             # módulos .orl (Layer 2/3)  
├─ runtime/            # libs empacotadas por triple  
├─ docs/spec/          # contrato normativo (EN)  
├─ docs/book/          # este livro (PT)  
├─ examples/           # programas .orl  
├─ extensions/         # VS Code + Zed (local)  
└─ tools/              # stage, smoke, QA, migrate
```

Como funciona

Pasta	Papel
compiler/	Workspace Cargo do compilador e do runtime em Rust
stdlib/	Fonte Ori da biblioteca padrão
runtime/<triple>/	Artefatos nativos para link/JIT
docs/spec/	Spec normativa
docs/guides/ + docs/language/	Guias e tour (EN + <code>.pt-BR.md</code>)
examples/	Amostras rodáveis
extensions/	LSP clients locais
tests/	Programas de teste E2E em Ori

Fluxo do desenvolvedor da linguagem (resumo):

1. Mudar crate ou `.orl` da stdlib
2. Testar (`cargo test`, `ori check` / exemplos)

3. Atualizar spec/CHANGELOG quando for user-visible
4. Re-estagiar runtime se a ABI/FFI mudou

O que memorizar

- Compilador em `compiler/`; `stdlib` em `stdlib/`; verdade normativa em `docs/spec/`.
- Este livro em `docs/book/` **não** substitui a spec.
- Exemplos vivos em `examples/` — use-os nos capítulos.

Ir mais fundo

- `../../../../AGENTS.md` (`../../../../AGENTS.md`) — contexto para agentes e mantenedores
- Apêndice B — mapa do repositório
- Cap. 8 — ferramentas do dia a dia

Cap. 5 — Da ideia à superfície S3

Versão âncora: Ori 0.3.x (S3)

Parte: II

TL;DR

A Ori manteve o **motor** (async, traits, ARC, nativo) e trocou a **pele** para S3 (ritmo Auk9). Em 0.3.0 o corte foi seco: pré-S3 morre. Em 0.3.1 veio inferência local; o pipe `|>` ficou. Auk9 como produto foi arquivada.

Exemplo

Norte em uma frase (registro de decisões):

Motor e features da Ori; pele e ritmo da Auk9; superfície vivente na Ori.

Como funciona

Marco	O que fechou
S3 / 0.3.0	Sintaxe canônica; rejeição dura do pré-S3
0.3.1	Inferência local Nim-style + opção B (campo/index/call/pipe)
Auk9	Lab/referência — não é mais produto paralelo
Pipe <code> ></code>	Mantido na Ori (Auk9 havia rejeitado)

Exceções conscientes Ori vs Auk9 pura (resumo):

- Alias: `import ori.io = io` (path → apelido)
- If-expressão: `if cond then a else b`
- Closures: `(u) => ...`
- Inferência local entregue; Auk9 v1 era mais rígida

Decisões detalhadas: blocos 0-9 em `../../planning/ori-surface-s3-auk9.md` (`../../planning/ori-surface-s3-auk9.md`).

O que memorizar

- S3 = pele; Ori = motor.
- Corte seco em 0.3.0 — sem dual longo.
- Pipe e inferência local B fazem parte da Ori vivente.

Ir mais fundo

- ADR: `../../planning/adr-ori-surface-s3-auk9.md` (`../../planning/adr-ori-surface-s3-auk9.md`)
- CHANGELOG `[0.3.0]` / `[0.3.1]` : `../../../CHANGELOG.md`
(`../../../CHANGELOG.md`)

Cap. 6 — Pipeline do compilador

Versão âncora: Ori 0.3.x (S3)

Parte: II

TL;DR

Fonte `.ori` passa por lexer, parser, resolução de nomes, type checker, HIR e codegen Cranelift. Depois: link AOT com `ori-runtime`, ou JIT em `ori run`.

Exemplo

```
Source (.ori)
  → Lexer          (ori-lexer)
  → Parser          (ori-parser) → AST
  → Resolver        (ori-hir)    → nomes / bindings
  → Type checker    (ori-types)
  → Codegen          (ori-codegen)
    └─ Native: Cranelift → objeto → link runtime
    └─ C debug: transpile parcial
  → Binary ou JIT in-process
```

Como funciona

Crates (ideia)	Papel
<code>ori-lexer</code>	Tokens
<code>ori-ast</code> / <code>ori-parser</code>	Árvore sintática
<code>ori-hir</code>	Resolução + HIR
<code>ori-types</code>	Tipos e diagnostics
<code>ori-codegen</code>	Cranelift / C
<code>ori-runtime</code>	ARC, I/O, FFI, async executor
<code>ori-diagnostics</code>	Códigos e render
<code>ori-driver</code>	CLI + testes de integração
<code>ori-lsp</code>	Language Server

AOT (`compile` / `test`): precisa de linker (bundled `rust-ld` ou linker do SO).

JIT (`run`): Cranelift in-process; símbolos `ori_*` da `cdylib` do runtime.

Há também backend C de debug com paridade parcial — útil para inspeção, não como caminho principal de aprendizado.

O que memorizar

- Ordem: lex → parse → resolve → types → codegen → link/JIT.
- Runtime é parte do produto (staticlib + cdylib).
- Diagnostics estáveis vivem no catálogo da spec 13.

Ir mais fundo

- Crates: `../../../../compiler/` (`../../../../compiler/`)
- Spec backend: `../../../../spec/14-backend-support.md` (`../../../../spec/14-backend-support.md`)
- Cap. 8 — ferramentas e env vars

Cap. 7 — Decisões de design

Versão âncora: Ori 0.3.x (S3)

Parte: II

TL;DR

Cada decisão grande da Ori responde a uma pergunta do leitor. Uma forma canônica por conceito. Sem inferência global (HM). Traits e recursos explícitos.

Exemplo

Checklist do manifesto antes de aceitar feature nova:

1. É visível na fonte onde o leitor precisa?
2. O contrato de tipo está legível no ponto de uso?
3. O erro ensina (código + ação)?
4. Há uma forma canônica simples?

Como funciona

Decisão	Por quê
<code>module</code> no topo	O arquivo declara onde mora
Tipos com <code>[]</code>	Uma gramática de tipos
<code>optional</code> / <code>result</code>	Ausência $\neq$ falha; ambos explícitos
Só <code>try</code>	Uma forma de propagar
<code>apply</code> + <code>use</code>	Origem do comportamento de trait visível
<code>using</code>	Cleanup determinístico e legível
Inferência local B	Menos ruído sem HM global
Pipe <code> ></code>	Composição legível da esquerda para a direita
Diagnostics com código	Erros como mapa, não como xingamento

O que **não** muda com a pele S3: poder do `async`, semântica forte de traits, ARC, capacidade da `stdlib`, rejeição de HM global, propósito do manifesto.

O que memorizar

- Uma forma canônica por conceito.
- Visível > mágico.
- Inferência local sim; HM global não.

Ir mais fundo

- Manifesto checklist: `../../spec/00-manifesto.md` (`../../spec/00-manifesto.md`)
- Specs `04 - 09`, `08-traits`

Cap. 8 — Ferramentas do dia a dia

Versão âncora: Ori 0.3.x (S3)

Parte: II

TL;DR

Desenvolver a Ori: Rust/Cargo, Cranelift, stage do runtime, testes do `ori-driver`, catálogo de diagnostics, LSP e scripts de smoke/package. Usar a Ori: CLI `ori` + editor com extensão local.

Exemplo

Ciclo mínimo no compilador:

```
cd compiler
cargo check --workspace
cargo test -p ori-driver --test ori_spec
cargo build -p ori-runtime --lib
# re-estagiar staticlib + cdylib em runtime/<triple>/ se a FFI mudou
```

Como funciona

Stack de implementação

Ferramenta	Uso
Rust + Cargo	Compilador e <code>ori-runtime</code>
Cranelift	Codegen nativo / JIT
logos	Lexer
tower-lsp	<code>ori-lsp</code>
Scripts <code>tools/stage_native_runtime.*</code>	Copiar runtime para <code>runtime/</code>
<code>tools/smoke_*</code>	Validar package isolado

CLI do usuário

`ori check` · `ori compile` · `ori run` · `ori test` · `ori migrate-syntax` · (e utilitários documentados no README / install).

Variáveis úteis (resumo)

Variável	Papel
<code>ORI_USE_JIT=1</code> / <code>ORI_USE_AOT=1</code>	Forçar caminho de <code>ori run</code>
<code>ORI_RUNTIME_LIB</code> / <code>ORI_RUNTIME_CDYLIB</code>	Override de artefatos
<code>ORI_USE_SYSTEM_LINKER=1</code>	Linker do SO
<code>ORI_STDLIB_ROOT</code>	Onde achar <code>stdlib/</code>
<code>UPDATE_EXPECT=1</code>	Atualizar snapshots de diagnostic em testes

Lista completa: `../../../../AGENTS.md` (`../../../../AGENTS.md`).

Editor

Extensões locais em `extensions/vscode-ori/` e `extensions/zed-ori/` — LanguageClient → `ori-lsp`. Sem prioridade de publish em marketplace no foco atual.

O que memorizar

- Mudou runtime FFI → re-estagiar **staticlib e cdylib**.
- Testes de linguagem: `ori-driver` + catálogo.
- Usuário final não precisa de Cargo.

Ir mais fundo

- Bootstrapping: `../../../../guides/bootstrapping.md` (`../../../../guides/bootstrapping.md`)
- Cap. 19 — CLI e env (consulta)

Cap. 9 — Desenvolver com assistência de IA

Versão âncora: Ori 0.3.x (S3)

Parte: II

TL;DR

A Ori é um lab explícito de **humano + agente** no mesmo código. Agentes seguem `AGENTS.md` e skills (compiler-dev, ori-testing, living-docs). Humanos fecham decisões de produto; a IA acelera implementação e docs — não inventa spec.

Exemplo

Contrato mental de uma sessão com agente:

1. Ler manifesto + spec do pedaço tocado
2. Implementar fatia pequena
3. Teste de regressão (`ori-driver` / exemplo)
4. CHANGELOG + docs no **mesmo** slice
5. Não reintroduzir sintaxe pré-S3

Como funciona

Papéis

Humano	Agente
Decide superfície e prioridades	Executa fatias com skills
Aceita ADR / corte de escopo	Propõe patches e testes
Faz review de contrato	Atualiza catálogo/CHANGELOG quando pedido

Skills do projeto (obrigatórias em código)

- `clean-code` — estrutura e nomes
- `rust` — crates e qualidade Rust
- `living-docs` — docs com o código
- `compiler-dev` — fase certa do front-end
- `lang-compiled` — AOT / IR / backends
- `ori-testing` — L1 check → L2 compile → L3 run
- `ori-lang-qa` — matriz e FREEZE

O que não pedir à IA

- “Aceitar de novo” sintaxe pré-S3
- Features de produto arquivadas (game/imgui como foco de linguagem)

- Spec inventada sem bater no compilador
- Self-hosting como próximo passo tático (M4 é último)

O que memorizar

- `AGENTS.md` é a constituição dos agentes neste repo.
- Teste + CHANGELOG + docs andam com o código.
- IA amplifica processo; não substitui o manifesto.

Ir mais fundo

- `../../../../AGENTS.md` (`../../../../AGENTS.md`)
- `../../../../CONTRIBUTING.md` (`../../../../CONTRIBUTING.md`)

Cap. 10 — Instalar e verificar o ambiente

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Usuário final instala o **package** de release (sem Rust). AOT precisa do linker do SO; `ori run` (JIT) só precisa do runtime empacotado. Confirme com `ori doctor` e um hello.

Exemplo

Após instalar e colocar `ori` no `PATH`:

```
ori --version
ori doctor
ori run examples/hello/main.ori
```

Ou, no diretório do exemplo:

```
cd examples/hello
ori run main.ori
```

`ori doctor` reporta saúde de stdlib, runtime, linker, target e JIT.

Como funciona

Pré-requisitos por SO (AOT)

SO	Precisa	Não precisa
Windows	VS Build Tools (C++)	Rust
Linux	<code>build-essential</code> (gcc + ld)	Rust
macOS	Xcode CLT	Rust

JIT (`ori run`): linker **não** é necessário se o `cdylib` do runtime estiver no package.

Caminhos de instalação

- Releases: GitHub Releases
- Guia completo: `../../install.pt-BR.md` (`../../install.pt-BR.md`)
- Windows (script): `tools/windows/get.ps1`
- Debian: `.deb` nas releases (`dpkg -i + build-essential` para AOT)

Checklist rápido se algo falhar

1. `ori --version` responde? → `PATH`
2. `ori doctor` reclama de runtime/stdlib? → package incompleto ou `ORI_STDLIB_ROOT`
3. `ori run` ok e `ori compile` falha? → linker do SO (AOT)
4. Só no clone do repo? → você precisa da toolchain Rust (Cap. 8 / bootstrapping)

Desenvolvedor do compilador

Clone o repo, Rust via `rust-toolchain.toml`, `cd compiler && cargo build -p ori-driver`.

Detalhe: `../../guides/bootstrapping.md` (`../../guides/bootstrapping.md`).

O que memorizar

- Package ≠ toolchain de desenvolvimento do compilador.
- `ori run` ≠ `ori compile` (JIT vs AOT).
- Primeiros sinais de vida: `ori doctor` + `ori run` no `examples/hello`.

Ir mais fundo

- Install PT: `../../install.pt-BR.md` (`../../install.pt-BR.md`)
- Primeiro projeto: `../../guides/first-project.pt-BR.md` (`../../guides/first-project.pt-BR.md`)
- Exemplo: `../../examples/hello` (`../../examples/hello`)
- Cap. 19 — CLI e env

Cap. 11 — Anatomia de um programa

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Todo arquivo começa com `module`. A entrada é `main()` **sem** keyword `func`. Blocos terminam com `end`. Imports usam `import path = alias` (path à esquerda).

Exemplo

```
module app.hello

import ori.io = io

main()
  io.print("Hello, Ori Language!")
  const name: string = "Developer"
  io.print(f"Welcome, {name}!")
end
```

(Fonte: `examples/hello/main.ori` (`../../examples/hello/main.ori`)).

Como funciona

Peça	Forma canônica
Cabeçalho	<code>module app.hello</code>
Import com apelido	<code>import ori.io = io</code>
Import seletivo	<code>import ori.fs (read_text)</code>
Função	<code>name(params) -> T / main()</code>
Bloco	indentação + <code>end</code>
Constante / variável	<code>const x: T = ... / var x: T = ...</code>
Comentário	<code>-- ...</code>

Inferência local (opção B)

Em `const / var` **locais**, o tipo pode ser omitido se o lado direito for literal óbvio, campo, índice, chamada com tipo conhecido ou pipe `|>`.

Na API pública (assinaturas), prefira tipos explícitos — o leitor agradece.

Formas que não existem mais (pré-S3)

`func`, `namespace`, `else if`, `?`, `import as` antigo, etc. → erro duro.
Migração mecânica: `ori migrate-syntax`.

O que memorizar

- `module` na primeira linha.
- Sem `func`; blocos com `end`.
- `import path = alias` — path à esquerda.

Ir mais fundo

- Tour: `../../language/tour.pt-BR.md` (`../../language/tour.pt-BR.md`)
- Spec statements: `../../spec/06-statements.md` (`../../spec/06-statements.md`)
- Cap. 15 — módulos e projetos

Cap. 12 — Tipos, ausência e falha

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Não há null. Ausência é `optional[T]` (`some / none`). Falha é `result[T, E]` (`ok / err`). Propagação só com `try expr`. Trate com `match` (ou `if some`).

Exemplo — `result` + `match`

```
module app.errors

import ori.io = io

divide(a: int, b: int) -> result[int, string]
  if b == 0
    return err("Division by zero")
  end
  return ok(a / b)
end

main()
  match divide(10, 2)
  case ok(val):
    io.print(f"ok: {val}")
  case err(msg):
    io.print(f"err: {msg}")
  end
end
```

(Ideia de `examples/error_handling` (`../../examples/error_handling/`)).

Exemplo — optional + try

```
module app.opt

import ori.io = io

find_user(id: int) -> optional[string]
  if id == 0
    return none
  end
  return some("alice")
end

greet(id: int) -> optional[string]
  const name: string = try find_user(id)
  return some(f"hi {name}")
end

main()
  match greet(1)
  case some(msg):
    io.println(msg)
  case none:
    io.println("missing")
  end
end
```

`try` em `optional` propaga `none`. Em `result`, propaga `err`.

Como funciona

Conceito	Papel	Construtores
<code>void</code>	Sem valor útil	—
<code>optional[T]</code>	Pode faltar	<code>some</code> / <code>none</code>
<code>result[T, E]</code>	Sucesso ou falha	<code>ok</code> / <code>err</code>
<code>try expr</code>	Propaga <code>none</code> ou <code>err</code>	única forma (sem <code>?</code>)
<code>check</code>	Invariante em runtime	aborta se falhar

Tipos compostos usam `[]`: `list[int]`, `map[string, int]`, `result[T, E]`.

Tratar vs propagar

Objetivo	Forma
Tratar no lugar	<code>match · if some(x) = expr</code>
Subir para o caller	<code>try expr</code> (função deve retornar <code>optional</code> ou <code>result</code> compatível)
Invariante “nunca deveria falhar”	<code>check cond, "msg"</code>

Escolha rápida

Situação	Use
Busca / EOF	<code>optional[T]</code>
I/O, validação	<code>result[T, E]</code>
Efeito sem valor	<code>void</code>
“Isso nunca pode acontecer”	<code>check</code>

Armadilhas comuns

- Usar `result` para “não achei” e `optional` para I/O — inverte os significados.
- Esperar `? pós-fix` — removido no S3.
- Esquecer que `main() -> result[...]` é necessário se `try` sobe até a entrada.

O que memorizar

- Sem `null` → `optional` ou `result`.
- Construtores: `ok/err`, `some/none`.
- Só `try` para propagar; `match` (ou `if some`) para tratar.

Ir mais fundo

- Guia: `../../guides/errors-null-void.pt-BR.md` (`../../guides/errors-null-void.pt-BR.md`)
- Spec: `../../spec/09-errors.md` (`../../spec/09-errors.md`), `../../spec/04-types.md` (`../../spec/04-types.md`)
- Exemplo: `../../examples/error_handling` (`../../examples/error_handling/`)

Cap. 13 — Expressões, fluxo e pipe

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Fluxo: `if` / `elif` / `else`, `while`, `for ... in`, `match`. If-expressão: `if cond then a else b`. Pipe `|>` tipa como `f(value)` e encadeia da esquerda para a direita.

Exemplo — fluxo + pipe

```
module app.flow

import ori.io = io

double(n: int) -> int => n * 2

main()
  const n: int = 3
  if n > 2
    io.println("big")
  elif n == 2
    io.println("two")
  else
    io.println("small")
  end

  const label: string = if n > 0 then "pos" else "non-pos"
  const doubled: int = n |> double
  io.println(f"{label} {doubled}")
end
```

Exemplo — encadeamento

Pipe aceita várias etapas (como em `examples/language_features` (`../../../../examples/language_features/`)):

```
-- ideia: 5 |> double |> add_ten |> to_label
const label: string = 5 |> double |> add_ten |> to_label
```

Cada etapa é uma função que recebe o valor à esquerda como **primeiro** argumento.

Como funciona

Forma	Notas
<code>if / elif / else</code>	Não use <code>else if</code>
<code>while cond ... end</code>	Loop condicional
<code>for x in xs ... end</code>	Iteração
<code>match</code>	<code>case ok(x): / case err(m): / enums</code>
<code>if c then a else b</code>	Expressão (valor)
<code>a > f</code>	Equivale a <code>f(a)</code>
<code>a > f > g</code>	<code>g(f(a))</code>

Inferência local B

Campo, índice, chamada e pipe no RHS de `const / var` local permitem omitir o tipo quando o retorno já é conhecido.

O que evitar

Evite	Use
<code>else if</code>	<code>elif</code>
Pipe com closure inventada sem testar	Função nomeada (<code>double</code>)
Confundir pipe com método encadeado estilo OO	É só açúcar de chamada

O que memorizar

- `elif`, nunca `else if`.
- Pipe = primeiro argumento à esquerda; encadeia com vários `|>`.
- Preferir `match` para `result / optional` ramificados.

Ir mais fundo

- Spec: `../../spec/05-expressions.md` (`../../spec/05-expressions.md`)
- Exemplo: `../../examples/language_features`
(`../../examples/language_features/`)

Cap. 14 — Structs, enums e funções

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Structs e enums declaram dados; literais usam `Type { campo: valor }`. Funções não usam `func`. Corpo curto pode usar `=>`. Generics usam `[]`.

Exemplo

```
module app.points

import ori.io = io

struct Point
  x: int
  y: int
end

add(a: int, b: int) -> int
  return a + b
end

double(n: int) -> int => n * 2

main()
  const p: Point = Point { x: 1, y: 2 }
  io.println(f"{p.x},{p.y} sum={add(p.x, p.y)} dup={double(p.x)}")
end
```

Como funciona

Structs

- Campos tipados; literal nomeado `Type { ... }`.
- Não há “struct call” estilo construtor posicional genérico pré-S3.

Enums

- Variantes; campos nomeados usam vírgulas dentro de parens (ver spec).
- `match` sem `.` nas variantes no padrão S3 do tour.

Funções

Forma	Uso
<code>name(params) -> T + end</code>	Corpo normal
<code>name(params) -> T => expr</code>	Corpo expressão
<code>main()</code>	Entrada

Generics: ver `../../spec/11-generics.md` (`../../spec/11-generics.md`) — tipos com `[]`.

Coleções do dia a dia: `list[T]`, `map[K, V]`, `set[T]` — ver `examples/collections_demo` (`../../examples/collections_demo/`).

O que memorizar

- Literal: `Type { f: v }`.
- Sem keyword `func`.
- `=>` para corpo curto.

Ir mais fundo

- Spec funções: `../../spec/07-functions.md` (`../../spec/07-functions.md`)
- Tour §6-7: `../../language/tour.pt-BR.md` (`../../language/tour.pt-BR.md`)

Cap. 15 — Módulos, imports e projetos

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Cada arquivo declara `module`. Imports: `import path = alias` ou seletivo. Projeto multiarquivo exige `ori.proj` com `entry` (raiz-first, sem `src/` obrigatório).

Exemplo — dois arquivos + manifest

`ori.proj` (mínimo útil; veja o exemplo real do repo):

```
manifest = 1
name = "greet_lab"
version = "0.1.0"
kind = "app"
entry = "main.orl"

[source]
root_namespace = "app"
```

```
-- greeter.orl
module app.greeter

public hello(name: string) -> string
    return f"Hello, {name}!"
end
```

```
-- main.orl
module app.main

import app.greeter = greeter
import ori.io = io

main()
    io.println(greeter.hello("Ori"))
end
```

```
ori check .
ori run main.orl
```

(Referência completa: `examples/multi_module`
(`../../examples/multi_module/`)).

Sem `ori.proj` / sem `entry`, `ori check` no diretório do projeto falha.

Como funciona

Imports

Forma	Efeito
<code>import ori.io = io</code>	Apelido local <code>io</code>
<code>import ori.fs (read_text)</code>	Nomes selecionados
<code>import ori.math</code>	Só <code>ori.math...</code> (sem apelido curto automático)

`import ori.io` **sozinho** não cria o nome local `io`.

Há também bloco `imports ... end` em alguns exemplos (vários aliases juntos). A forma de linha `import path = alias` é a canônica do tour e deste livro.

Visibilidade

`public` em itens que outros módulos do projeto devem importar (como `hello` acima).

Projeto

```
my_app/  
  ori.proj  
  main.orl
```

```
ori new my_app  
ori run main.orl
```

Layout e contrato: `../../spec/17-project-and-docs.md` (`../../spec/17-project-and-docs.md`).

O que memorizar

- Path do import à esquerda do `=`.
- Prefira país `ori.X`, não `.utils` em material novo.
- Multiarquivo → `ori.proj` com `entry`.

Ir mais fundo

- Guia: `../../guides/first-project.pt-BR.md` (`../../guides/first-project.pt-BR.md`)
- Exemplo: `../../examples/multi_module` (`../../examples/multi_module/`)

Cap. 16 — Traits e comportamento explícito

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Comportamento compartilhado vem de traits. Sintaxe: `apply Type` com seções `use Trait`. Importe o módulo do trait (`ori.core`). Sem `implement ... for`.

Exemplo

```
module app.points

import ori.core = core
import ori.io = io

struct Point
  x: int
  y: int
end

apply Point
  use core.Displayable
  display(self) -> string
    return f"({self.x}, {self.y})"
  end
end

main()
  const p: Point = Point { x: 1, y: 2 }
  io.println(string(p))
end
```

Como funciona

- **Pele S3, motor Ori:** traits de verdade (bounds, monomorph, defaults por corpo).
- `apply Type` agrupa as implementações daquele tipo.
- `use Trait` declara qual contrato está sendo preenchido.
- Conversão comum: `string(value)` via `Displayable`, não método solto mágico fora do trait.
- Default de método: corpo na definição do trait (sem keyword `default` da Auk9).

O que memorizar

- `apply` + `use`, não `implement`.
- Importe `ori.core` (ou o módulo dono do trait).
- Origem do comportamento fica visível na fonte.

Ir mais fundo

- Spec: `../../spec/08-traits.md` (`../../spec/08-traits.md`)
- Tour §6: `../../language/tour.pt-BR.md` (`../../language/tour.pt-BR.md`)

Cap. 17 — Stdlib, I/O, async e testes

Versão âncora: Ori 0.3.x (S3)

Parte: III

TL;DR

Stdlib canônica: `ori.X`. Layer 1 (Rust/runtime) para hot path; Layer 2/3 em `.ori`. I/O e FS via imports explícitos. Async no backend nativo. Testes com `ori test`.

Exemplo — FS + `result`

```
module app.main

import ori.fs = fs
import ori.io = io

main() -> result[void, string]
  const text: string = try fs.read_text("notes.txt")
  io.println(text)
  return ok()
end
```

Exemplo — teste mínimo

```
module app.main

import ori.test = test

@test
math_is_stable()
  test.assert(1 + 1 == 2, "math should work")
end
```

```
ori test
```

(Com `ori.proj` + `entry`. Ver também `examples/tests_demo`
(`../../../../examples/tests_demo/`)).

Como funciona

Camadas

Layer	Onde	Papel
1	Runtime Rust + manifest	FFI, ARC, FS/net quentes
2/3	<code>stdlib/*.orl</code>	Ergonomia e algoritmos em Ori

API pública nova: só em `ori.X.Paths` `ori.X.utils` são compat silenciosa — não ensinar como API nova.

Domínios úteis

`ori.io`, `ori.fs`, `ori.string`, `ori.bytes`, `ori.net`, `ori.os`, `ori.process`, `ori.json`, `ori.test`, ... — índice no Cap. 20.

Async (nativo)

- Entrada: `async main()` quando o programa é assíncrono.
- APIs awaitable: sufixos `*_async` / futures da `stdlib` (FS/net conforme spec).
- Backend C/debug **rejeita** `async` — use o caminho nativo.
- Rede (`examples/http_get`) precisa de ambiente/rede real; não trate como unitário puro.

`using`

Para recursos com cleanup determinístico e visível — ver `examples/using_fs` (`../../examples/using_fs/`) e a spec de memória/erros.

Exemplos no repo

Exemplo	Tema
<code>using_fs</code> / <code>path_time_io</code>	FS e tempo
<code>async_demo</code> / <code>concurrency</code>	Async / concorrência
<code>tests_demo</code>	Testes
<code>http_get</code>	Rede (avançado)

Mantenedor vs usuário

Você é...	Como testar
App Ori	<code>ori test</code> no projeto
Compilador	<code>cargo test</code> + testes <code>ori-driver</code>

O que memorizar

- `import ori.X = alias` — pais canônicos.
- Layer 1 Rust é design permanente, não “dívida”.
- Async = backend nativo; rede = ambiente real.

Ir mais fundo

- Spec stdlib: `../../spec/12-stdlib.md` (`../../spec/12-stdlib.md`)
- `../../stdlib/README.md` (`../../stdlib/README.md`)
- Testing: `../../guides/testing.pt-BR.md` (`../../guides/testing.pt-BR.md`)
- Cap. 20 — índice de consulta

Cap. 18 — Cheat sheet de sintaxe S3

Versão âncora: Ori 0.3.x (S3)

Parte: IV

Tabela compacta também em **Apêndice A** (`../apendices/A-cheat-sheet.md`).

TL;DR

Uma página para consultar formas canônicas e rejeições pré-S3. Em dúvida, `ori check` e o catálogo de erros (Cap. 21).

Forma válida (resumo)

Ver **Apêndice A** (`../apendices/A-cheat-sheet.md`) — mesma tabela.

Programa mínimo

```
module app.main

import ori.io = io

main()
  io.println("ok")
end
```

Migração

```
ori migrate-syntax caminho/
```

Ir mais fundo

- Overview: `../../spec/01-overview.md` (`../../spec/01-overview.md`)
- Catálogo: `../../spec/13-error-catalog.md` (`../../spec/13-error-catalog.md`)

Cap. 19 — CLI e variáveis de ambiente

Versão âncora: Ori 0.3.x (S3)

Parte: IV

TL;DR

Comandos do dia: `check`, `compile`, `run`, `test`. `run` tende a JIT; `compile` e `test` são AOT. Overrides via `ORI_*` quando precisar diagnosticar link/runtime.

Comandos

Comando	Papel
<code>ori check <path></code>	Parse + tipos (rápido)
<code>ori compile <path></code>	AOT → binário
<code>ori run <path></code>	Executa (JIT se cdylib disponível)
<code>ori test</code>	Suite de testes do projeto
<code>ori doctor</code>	Saúde de stdlib, runtime, linker, target, JIT
<code>ori migrate-syntax <path></code>	Reescreve várias formas pré-S3
<code>ori new <name></code>	Esqueleto de projeto

Variáveis (consulta prática)

Variável	Quando usar
<code>ORI_USE_AOT=1</code>	Forçar AOT em <code>ori run</code>
<code>ORI_USE_JIT=1</code>	Forçar JIT
<code>ORI_RUNTIME_LIB</code>	Staticlib fora do lugar padrão
<code>ORI_RUNTIME_CDYLIB</code>	Cdylib para JIT
<code>ORI_STDLIB_ROOT</code>	Stdlib fora do package
<code>ORI_USE_SYSTEM_LINKER=1</code>	Linker do SO
<code>ORI_USE_BUNDLED_RUST_LLD=1</code>	<code>rust-ld</code> empacotado/descoberto

Lista completa para mantenedores: `../../../../AGENTS.md` (`../../../../AGENTS.md`).

O que memorizar

- Check antes de compile quando só quer tipos.

- JIT $\neq$ AOT — sintomas de “funciona no run e falha no compile” costumam ser linker/runtime.
- Usuário de package raramente precisa de `ORI_*`.

Ir mais fundo

- Install: `../../install.pt-BR.md` (`../../install.pt-BR.md`)
- Cap. 8 — ferramentas do compilador

Cap. 20 — Stdlib: índice de consulta

Versão âncora: Ori 0.3.x (S3)

Parte: IV

TL;DR

Importe pais `ori.X`. Abaixo: mapa de um olhada. Detalhe normativo na spec 12; fontes em `stdlib/*.orl` + Layer 1 no runtime.

Índice (propósito em uma linha)

Módulo	Propósito
<code>ori.io</code>	Print / eprint / streams básicos
<code>ori.fs</code>	Arquivos e diretórios
<code>ori.path</code>	Manipulação de caminhos
<code>ori.string</code>	Texto
<code>ori.bytes</code>	Binário
<code>ori.list</code> / <code>ori.map</code> / <code>ori.set</code>	Coleções
<code>ori.iter</code>	Iteração
<code>ori.math</code>	Matemática
<code>ori.time</code>	Tempo
<code>ori.os</code>	SO / processo ambiente
<code>ori.process</code>	Subprocessos
<code>ori.args</code>	Argumentos de CLI
<code>ori.net</code>	TCP/UDP/TLS (rede)
<code>ori.json</code>	JSON
<code>ori.log</code>	Logging
<code>ori.random</code>	Aleatório
<code>ori.test</code>	Asserções / helpers de teste
<code>ori.core</code>	Traits e núcleo (<code>Displayable</code> , ...)
<code>ori.format</code>	Formatação
<code>ori.convert</code>	Conversões
<code>ori.config</code>	Config
<code>ori.validate</code>	Validação
<code>ori.crypto</code>	Cripto (quando disponível)
<code>ori.concurrent</code> / filas / heaps / graphs	Estruturas e concorrência

Tipos fundamentais (`optional` , `result` , `list` , ...) existem sem import.

Import canônico

```
import ori.io = io
import ori.fs = fs
import ori.string = str
```

Evite ensinar `ori.*.utils` / `ori.*.algorithms` como API nova.

Ir mais fundo

- Spec: `../../spec/12-stdlib.md` (`../../spec/12-stdlib.md`)
- `../../../stdlib/README.md` (`../../../stdlib/README.md`)
- Cap. 17 — uso prático

Cap. 21 — Diagnostics mais comuns

Versão âncora: Ori 0.3.x (S3)

Parte: IV

TL;DR

Todo diagnostic tem um **código** estável. Leia o código + a mensagem + a ação sugerida. Catálogo completo: spec 13. Aqui: os que mais aparecem no dia a dia.

Como ler um erro

1. **Código** — família (`name.*`, `bind.*`, `type.*`, ...)
2. **Span** — onde no arquivo
3. **Mensagem** — o que falhou
4. **Nota / help** — o que tentar

Famílias frequentes

Prefixo	Significado típico
<code>name.*</code>	Nome não resolvido, privado, duplicado no topo
<code>bind.*</code>	Binding/import/campo/param (ex. <code>import stdlib</code> desconhecido)
<code>type.*</code>	Incompatibilidade de tipos
(códigos de forma pré-S3)	Sintaxe removida — migrar

Convenção importante: o emitido para “não definido” é `name.undefined` (`bind.undefined` é alias reservado no catálogo).

Exemplo real

Código sem import:

```
module app.ex
main()
  io.println("boom")
end
```

`ori check` emite algo como:

```
error[name.undefined]: undefined name `io.println`  
  = action: declare or import the name before using it
```

Correção: `import ori.io = io.`

Hábitos úteis

- `ori check` primeiro — barato.
- Se veio de código antigo: `ori migrate-syntax.`
- Em testes do compilador: catálogo deve bater com `diagnostic_catalog_matches_emitted_codes.`

Ir mais fundo

- Catálogo: `../../spec/13-error-catalog.md` (`../../spec/13-error-catalog.md`)
- Reportar bugs: `../../guides/report-bugs.pt-BR.md` (`../../guides/report-bugs.pt-BR.md`)

Cap. 22 — Estabilidade, ABI e limites

Versão âncora: Ori 0.3.x (S3)

Parte: IV

TL;DR

Ori está em **0.3.x** (pré-1.0) com FREEZE-1: mudança ainda possível, com disciplina. ABI nativa documentada como `ori-native-abi-1`. Self-hosting (M4) não é o próximo passo tático.

Estabilidade

Tema	Estado
Superfície S3	Canônica desde <code>0.3.0</code> ; pré-S3 rejeitado
Inferência B	Desde <code>0.3.1</code>
Package / install sem Rust	M1 fechado
ABI documentada	M3 — <code>../.. /spec/19-abi.md</code> (<code>../.. /spec/19-abi.md</code>)
Self-host	M4 — última discussão
1.0	Critério de maturidade (anos), não de marketing

Ver: `../.. /spec/18-stability-and-compatibility.md` (`../.. /spec/18-stability-and-compatibility.md`).

Limites e pitfalls (mantenedor / usuário avançado)

1. Runtime desatualizado → `native.link_failed` — re-estagiar staticlib e cdylib.
2. Cache `OnceLock` de path de runtime em testes — primeiro resultado “gruda”.
3. Windows: `ori-lsp.exe` pode travar rebuild — encerrar o processo.
4. Backend C: paridade parcial; async é caminho nativo.
5. Rede/TLS: precisa de ambiente; exemplos como `http_get` não são unitários puros.

O que memorizar

- Livro âncora = 0.3.x; leia o CHANGELOG ao atualizar.
- ABI ≠ estabilidade de sintaxe 1.0.
- M4 self-host não bloqueia uso do lab hoje.

Ir mais fundo

- ABI: `../../../../spec/19-abi.md` (`../../../../spec/19-abi.md`)
- Backend: `../../../../spec/14-backend-support.md` (`../../../../spec/14-backend-support.md`)
- Backlog: `../../../../planning/BACKLOG.md` (`../../../../planning/BACKLOG.md`)

Apêndice A — Cheat sheet de sintaxe S3

Versão âncora: Ori 0.3.x

Espelho expandido: **Cap. 18** (../parte-4/18-cheat-sheet-sintaxe.md)

Forma válida

Tema	Forma
Módulo	<code>module app.main</code>
Função	<code>name(x: int) -> int · main() — sem func</code>
Bloco	<code>corpo + end</code>
Tipos	<code>list[T], optional[T], result[T, E] com []</code>
Result	<code>ok(v) / err(e) · case ok(x): / case err(m):</code>
Optional	<code>some(v) / none</code>
Propagar	<code>try expr apenas</code>
Import alias	<code>import ori.io = io (path à esquerda)</code>
Import seletivo	<code>import ori.fs (read_text)</code>
If	<code>if / elif / else (não else if)</code>
If-expressão	<code>if cond then a else b</code>
Struct literal	<code>Point { x: 1, y: 2 }</code>
Traits	<code>apply Type + use Trait</code>
Pipe	<code>value > f → f(value)</code>
Cleanup	<code>using ... end</code>
Comentário	<code>-- ...</code>

Rejeitado (pré-S3 → erro duro)

Evite	Use
<code>func / namespace</code>	<code>name() / module</code>
<code>else if</code>	<code>elif</code>
<code>?</code>	<code>try</code>
<code>import as / ordem Auk9 invertida</code>	<code>import path = alias</code>
<code><></code> em tipos	<code>[]</code>
<code>implement / apply Trait to</code>	<code>apply Type + use Trait</code>
<code>success / error</code>	<code>ok / err</code>
<code>do</code> em closures	<code>(u) => ...</code>

Migração: `ori migrate-syntax`.

CLI mínima

```
ori check path.orl
ori compile path.orl
ori run path.orl
ori test
```

Apêndice B — Mapa do repositório

Uma página. Detalhe narrativo: **Cap. 4** (`../parte-1/04-mapa-ecossistema.md`).

```
ori-lang/
├─ compiler/crates/    ori-lexer, ori-parser, ori-ast, ori-hir, ori-types,
│                      ori-codegen, ori-runtime, ori-diagnostics,
│                      ori-driver, ori-lsp
├─ stdlib/             módulos ori.* (.orl)
├─ runtime/<triple>/   staticlib + cdylib empacotados
├─ docs/
│   ├─ spec/           contrato normativo (EN)
│   ├─ language/       tour
│   ├─ guides/         how-tos
│   ├─ planning/       mantenedores
│   └─ book/           este livro (PT)
├─ examples/          programas .orl
├─ tests/             E2E Ori
├─ extensions/        vscode-orl, zed-ori
├─ tools/             stage, smoke, QA, migrate
├─ branding/         logo
└─ CHANGELOG.md
```

Apêndice C — Exercícios

Soluções sugeridas em `../exercicios/` (`../exercicios/`).

Use `ori check` / `ori run` em cada um.

1. **Hello pessoal** — Imprima seu nome com `f"..."`. (Caps 10-11)
2. **Soma tipada** — Função `add(a: int, b: int) -> int`. (Cap. 14)
3. **Divisão segura** — `divide` retorna `result[int, string]`; trate com `match`. (Cap. 12)
4. **Optional** — `find` retorna `optional[string]`; use `if some(x) = ...`. (Cap. 12)
5. **Pipe** — Encadeie duas funções com `|>`. (Cap. 13)
6. **Struct Point** — Literal + print dos campos. (Cap. 14)
7. **Dois módulos** — `greeter` + `main` no mesmo projeto. (Cap. 15)
8. **Displayable** — `apply` + use `core.Displayable` e `string(p)`. (Cap. 16)
9. **Ler arquivo** — `fs.read_text` + `try` em `main()` -> `result[...]`. (Cap. 17)
10. **Teste** — Um `ori.test` assert simples (ver `examples/tests_demo`). (Cap. 17)
11. **CLI args** — Leia um argumento e imprima (ver `examples/cli_args`). (Caps 11, 19)
12. **Diagnostic** — Provocar `name.undefined` de propósito e ler o código. (Cap. 21)

Apêndice D — Programa em 30 minutos

Roteiro único: dobrar um número com pipe e tratar divisão com `result`.

Minutos 0-5 — ambiente

```
ori --version
ori doctor
ori new double_lab
cd double_lab
```

Minutos 5-15 — código

Substitua o corpo de `main.orl` por:

```
module app.main

import ori.io = io

double(n: int) -> int => n * 2

safe_div(a: int, b: int) -> result[int, string]
  if b == 0
    return err("zero")
  end
  return ok(a / b)
end

main()
  const raw: int = 10
  const n: int = raw |> double
  match safe_div(n, 2)
    case ok(v):
      io.println(f"ok {v}")
    case err(m):
      io.println(f"err {m}")
  end
end
```

Minutos 15-25 — rodar

```
ori check main.orl
ori run main.orl
```

Saída esperada: algo como `ok 10 (20 / 2)`.

Minutos 25-30 — estender

1. Troque o denominador por `0` e veja o `err`.
2. Extraia `safe_div` para outro arquivo e importe (Cap. 15 — lembre do `ori.proj`).
3. Opcional: leia um inteiro via `examples/cli_args` (`../../../../examples/cli_args/`).

Ir mais fundo

- Caps 12, 13, 15
- `../../../../guides/first-project.pt-BR.md` (`../../../../guides/first-project.pt-BR.md`)

Apêndice E — Bibliografia

Interna (repositório)

Recurso	Uso
<code>../../spec/00-manifesto.md</code> (<code>../../spec/00-manifesto.md</code>)	Identidade
<code>../../spec/</code> (<code>../../spec/</code>)	Contrato normativo
<code>../../language/tour.pt-BR.md</code> (<code>../../language/tour.pt-BR.md</code>)	Tour curto
<code>../../guides/</code> (<code>../../guides/</code>)	How-tos
<code>../../planning/ori-surface-s3-auk9.md</code> (<code>../../planning/ori-surface-s3-auk9.md</code>)	Decisões S3
<code>../../CHANGELOG.md</code> (<code>../../CHANGELOG.md</code>)	Histórico de versões
<code>../../AGENTS.md</code> (<code>../../AGENTS.md</code>)	Contexto para agentes/mantenedores
<code>../../examples/</code> (<code>../../examples/</code>)	Programas rodáveis

Externa (curta)

Sugestões gerais — não são dependências do projeto:

- Textbooks clássicos de compiladores (lexing/parsing/IR) para acompanhar a Parte II
- Materiais sobre carga cognitiva e plain language (ex. W3C COGA) para a Parte I
- Documentação Cranelift / linkers do SO se for aprofundar Cap. 6 e 8

O livro Ori **não** substitui esses textos; usa a Ori como caso de estudo.

Glossário

Termos usados no livro. Definição curta; detalhe na spec ou no capítulo citado.

Termo	Significado
AOT	Ahead-of-time: compila para binário nativo antes de executar (<code>ori compile</code>)
JIT	Just-in-time: executa código Cranelift in-process (<code>ori run</code> quando há <code>cdylib</code>)
S3	Superfície de sintaxe canônica desde 0.3.0 (estilo poema / Auk9-inspired)
HIR	High-level IR — representação intermediária após resolução de nomes
ARC	Automatic Reference Counting — gestão de memória do runtime Ori
Diagnostic	Mensagem de erro/aviso com código estável (ex. <code>name.undefined</code>)
Stdlib Layer 1	Funções hot-path implementadas no runtime Rust (FFI)
Stdlib Layer 2/3	Módulos <code>.orl</code> na pasta <code>stdlib/</code>
ABI	Application Binary Interface — contrato nativo (<code>ori-native-abi-1</code>)
FREEZE-1	Política de estabilidade em 0.3.x (mudança ainda possível, com disciplina)
Pipe <code> \></code>	Operador que passa o valor à esquerda como primeiro argumento da chamada
Inferência local (opção B)	Omitir tipo em <code>const / var</code> local quando o RHS é óbvio
Trait	Contrato de comportamento; em Ori via <code>apply Type + use Trait</code>
optional[T]	Valor presente (<code>some</code>) ou ausente (<code>none</code>) — sem null
result[T, E]	Sucesso (<code>ok</code>) ou falha (<code>err</code>)
using	Escopo que libera recurso de forma determinística e visível
Auk9	Lab de sintaxe arquivado; a pele vivente está na Ori S3

Exemplos canônicos do livro

Prefira estes programas do repositório. Não invente exemplos órfãos quando já existir um `examples/` estável.

Exemplo	Capítulos	O que mostra
<code>examples/hello</code> (<code>../../examples/hello/</code>)	10, 11	<code>module</code> , <code>main</code> , <code>print</code> , <code>const</code>
<code>examples/error_handling</code> (<code>../../examples/error_handling/</code>)	12	<code>result</code> , <code>ok</code> / <code>err</code> , <code>try</code> , <code>match</code>
<code>examples/language_features</code> (<code>../../examples/language_features/</code>)	13, 14	superfície da linguagem
<code>examples/collections_demo</code> (<code>../../examples/collections_demo/</code>)	14	listas / coleções
<code>examples/multi_module</code> (<code>../../examples/multi_module/</code>)	15	vários arquivos
<code>examples/using_fs</code> (<code>../../examples/using_fs/</code>)	17	FS / <code>using</code>
<code>examples/path_time_io</code> (<code>../../examples/path_time_io/</code>)	17	<code>path</code> , <code>time</code> , I/O
<code>examples/async_demo</code> (<code>../../examples/async_demo/</code>)	17	async básico
<code>examples/concurrency</code> (<code>../../examples/concurrency/</code>)	17	concorrência
<code>examples/tests_demo</code> (<code>../../examples/tests_demo/</code>)	17	testes
<code>examples/http_get</code> (<code>../../examples/http_get/</code>)	17	rede (requer ambiente)
<code>examples/cli_args</code> (<code>../../examples/cli_args/</code>)	11, 19	argumentos de CLI

Regra

Snippets no corpo do capítulo: **≤ 15 linhas**.

Programas maiores: link para `examples/` ou apêndice.

Soluções sugeridas dos exercícios

Enunciados: **Apêndice C** (`../apendices/C-exercicios.md`).

Todos os trechos abaixo passaram por revisão com a superfície S3 em mente; rode `ori check` no seu ambiente ao adaptar.

1 — Hello pessoal

```
module app.ex1

import ori.io = io

main()
  const name: string = "Raillen"
  io.println(f"Hello, {name}!")
end
```

2 — Soma tipada

```
module app.ex2

import ori.io = io

add(a: int, b: int) -> int
  return a + b
end

main()
  io.println(f"{add(2, 3)}")
end
```

3 — Divisão segura

```
module app.ex3

import ori.io = io

divide(a: int, b: int) -> result[int, string]
  if b == 0
    return err("zero")
  end
  return ok(a / b)
end

main()
  match divide(10, 0)
  case ok(v):
    io.println(f"{v}")
  case err(m):
    io.println(m)
  end
end
```

4 — Optional

```
module app.ex4

import ori.io = io

find(id: int) -> optional[string]
  if id == 1
    return some("ok")
  end
  return none
end

main()
  if some(name) = find(1)
    io.println(name)
  else
    io.println("missing")
  end
end
```

5 — Pipe

```
module app.ex5

import ori.io = io

double(n: int) -> int => n * 2
inc(n: int) -> int => n + 1

main()
  const v: int = 3 |> double |> inc
  io.println(f"{v}")
end
```

6 — Struct Point

```
module app.ex6

import ori.io = io

struct Point
  x: int
  y: int
end

main()
  const p: Point = Point { x: 1, y: 2 }
  io.println(f"{p.x},{p.y}")
end
```

7 — Dois módulos

Use o esqueleto do **Cap. 15** (`../parte-3/15-modulos-projetos.md`) (`ori.proj` + `greeter.ori` + `main.ori`) ou copie `examples/multi_module` (`../../examples/multi_module/`).

8 — Displayable

Espehe o exemplo completo do **Cap. 16** (`../parte-3/16-traits.md`).

9 — Ler arquivo

Espehe o exemplo FS do **Cap. 17** (`../parte-3/17-stdlib-io-async-testes.md`) (crie `notes.txt` ao lado ou trate o `err`).

10 — Teste

Siga `examples/tests_demo` (`../../examples/tests_demo/`) e `../guides/testing.pt-BR.md` (`../guides/testing.pt-BR.md`).

11 — CLI args

Siga `examples/cli_args` (`../../../examples/cli_args/`) (usa bloco `imports ...` `end` e `ori.args`).

12 — Diagnostic

```
module app.ex12

main()
  io.println("boom")
end
```

`ori check` deve apontar algo na família `name.*` (nome `io` indefinido).
Depois corrija com `import ori.io = io` .